

Yesterday 3:38 PM

Java Scenario-Based Questions

These are where 5-year interviews become different from 2–3 year interviews.

Scenario 1

Your application suddenly becomes slow after deployment. CPU is normal but response time has increased. How will you investigate?

Show more ▾



Absolutely. For a **5-year Java developer**, these questions are less about syntax and more about **how you think in production**: identify the bottleneck, isolate the problem, choose the right Java construct, and explain trade-offs.

Below are **interview-ready answers**. Try to answer each in this structure:

Understand → Measure → Identify bottleneck → Fix → Prevent recurrence

Java Scenario-Based Interview Questions — 5 Years Experience

Scenario 1: Application suddenly becomes slow after deployment

Question:

Your application suddenly becomes slow after deployment. CPU is normal but response time has increased. How will you investigate?

Strong 5-year answer

I would first determine whether the latency is coming from the **application itself or a downstream dependency**.

My investigation would be:

1. Compare before vs after deployment

I would check:

- Response time before deployment
- Response time after deployment
- Error rate
- Throughput
- p50/p95/p99 latency

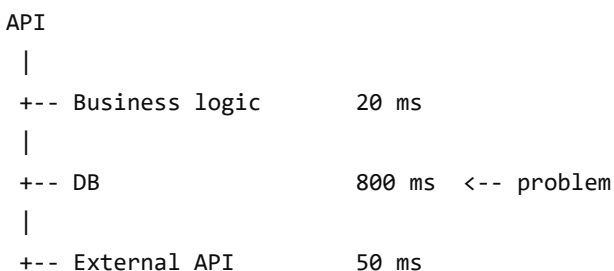
If only latency increased while CPU is normal, I would not immediately assume a CPU problem.

2. Check application metrics

I would look at:

- DB latency
- External API latency
- Connection pool usage
- Thread pool utilization
- Queue size
- GC pauses/behavior at the application level
- HTTP client connection pool
- Lock contention
- Request timeout count

For example:



```
API
|
+-- Business logic      20 ms
|
+-- DB                  800 ms  <-- problem
|
+-- External API       50 ms
```

The application CPU can be completely normal while requests are waiting on the DB.

3. Check distributed tracing

If we use something such as OpenTelemetry/Micrometer tracing, I would trace a slow request:



```
API
|
+-- Controller      10ms
|
+-- Service         20ms
|
+-- DB Query        900ms
|
+-- External API    100ms
```

This immediately helps identify where the latency is being introduced.

4. Check the deployment changes

Because the problem started **after deployment**, I would compare:

- Code changes
- SQL changes
- Configuration changes
- Timeout changes
- Connection pool configuration
- Thread pool configuration
- New downstream calls
- New logging
- Feature flags

5. Check database queries

A common example is:



```
</> Java

repository.findUsers();
```

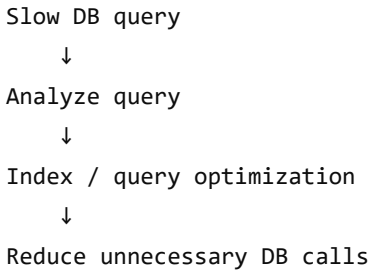
Before deployment it might execute one query.

After a code change, it could accidentally cause many additional queries.

I would check query latency and query count.

6. Fix based on evidence

For example:



```
graph TD; A[Slow DB query] --> B[Analyze query]; B --> C[Index / query optimization]; C --> D[Reduce unnecessary DB calls];
```

I wouldn't blindly increase threads or CPU without identifying the bottleneck.

Interview closing statement

"Since CPU is normal, I would focus on I/O wait, database latency, downstream services, thread/connection pool saturation, locks, and configuration changes. I would use metrics and distributed tracing to identify the exact latency contributor before making changes."

Scenario 2: 1,000 requests/sec and each request takes 2 seconds

Question:

An API receives 1,000 requests/sec and each request takes 2 seconds. How would you improve it?

First, I would understand **why each request takes 2 seconds**.

The important question isn't simply:

"How many threads do I need?"

It is:

"What is the request waiting for?"

Step 1: Measure

Suppose:

1000 requests/sec
Average latency = 2 sec



Approximately:

Concurrency $\approx$ throughput $\times$ latency
 $\approx 1000 \times 2$
 ≈ 2000 concurrent requests



That means the system potentially needs to handle around **2,000 in-flight requests**.

But increasing threads isn't automatically the solution.

Step 2: Find the bottleneck

Possible causes:

API
|
+-- CPU processing
|
+-- Database
|
+-- External API
|
+-- Lock
|
+-- Queue



If the 2 seconds comes from a database call, optimize the database.

If it comes from a downstream API, use:

- Timeout
- Connection pooling
- Caching where appropriate
- Concurrent calls where independent
- Circuit breaker
- Bulkhead

Step 3: Parallelize independent work

Bad:

</> Java

```
resultA = callA();  
resultB = callB();  
resultC = callC();
```



If each takes 500 ms:

Total $\approx$ 1500 ms



If independent:

</> Java

```
CompletableFuture<A> a = ...  
CompletableFuture<B> b = ...  
CompletableFuture<C> c = ...
```



Then wait for all.

Potentially:

500 ms



instead of:

1500 ms



Step 4: Remove unnecessary work

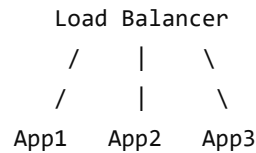
Look for:

- N+1 queries
- Repeated DB calls
- Expensive serialization
- Unnecessary external calls
- Duplicate calculations
- Excessive logging

- Large payloads

Step 5: Scale appropriately

If the application is stateless:



We can horizontally scale instances.

But scaling doesn't fix a saturated database.

Strong interview answer

"At 1,000 RPS and 2-second latency, I would first identify the reason for the 2 seconds. I would use metrics and tracing to determine whether the bottleneck is CPU, DB, downstream APIs, locking, or I/O. Then I would optimize the bottleneck, parallelize independent operations, introduce caching where appropriate, tune connection/thread pools based on measurements, and horizontally scale stateless instances if required."

Scenario 3: Two threads update the same account balance

Question:

Two threads are updating the same account balance. How do you prevent incorrect results?

This is a **race condition**.

Suppose:

Initial balance = 1000



Thread A:

```
read 1000
+100
```



```
write 1100
```

Thread B:

```
read 1000  
-200  
write 800
```



One update can overwrite the other.

Option 1: synchronized

</> Java



```
public synchronized void updateBalance(int amount) {  
    balance += amount;  
}
```

Only one thread executes the critical section at a time.

Option 2: Lock

</> Java



```
private final Lock lock = new ReentrantLock();  
  
public void updateBalance(int amount) {  
    lock.lock();  
    try {  
        balance += amount;  
    } finally {  
        lock.unlock();  
    }  
}
```

Useful when we need more flexible locking behavior.

Option 3: AtomicInteger

For a simple atomic numeric update:

</> Java



```
private final AtomicInteger balance = new AtomicInteger(1000);

balance.addAndGet(amount);
```

But for real banking/account operations, the problem is often bigger than one variable.

If the balance is persisted in a database, I would consider **transactional consistency and database concurrency control** too.

For example:

</> SQL



```
UPDATE account
SET balance = balance + ?
WHERE account_id = ?;
```

possibly combined with an appropriate transaction/locking strategy.

Strong interview answer

"If the balance is only in memory, I can protect the update using synchronized, Lock, or AtomicInteger depending on the operation. If this is a persistent account balance, I also need transaction-level concurrency control because JVM synchronization only protects threads inside one JVM instance."

That last sentence is **very important for a 5-year candidate**.

Scenario 4: Five downstream services, one is slow

Question:

Your application calls five downstream services. One service is slow. How do you prevent it from affecting the entire application?

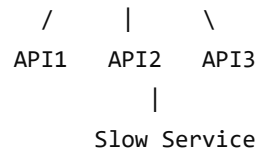
I would use **timeouts + circuit breaker + bulkhead**, and possibly fallback.

Architecture:

Main Service

/ | \





1. Timeout

Never allow the request to wait indefinitely.

Downstream timeout = 500ms



If the service doesn't respond:

fail fast



2. Circuit breaker

If the service repeatedly fails:

CLOSED
↓ failures
OPEN
↓
Reject calls immediately



After some time:

HALF_OPEN



Test whether the service recovered.

Then:

HALF_OPEN → CLOSED



if healthy.

3. Bulkhead

Don't allow one dependency to consume all resources.

For example:

Thread Pool



Service A → 20 threads
Service B → 20 threads
Service C → 20 threads
Slow Service → 10 threads

The slow service cannot consume every available thread.

4. Fallback

If business requirements allow:

</> Java



```
try {  
    return downstreamService.getData();  
} catch (TimeoutException e) {  
    return defaultResponse();  
}
```

But fallback should only be used when returning stale/default/partial data is acceptable.

Strong answer

"I would isolate the unreliable dependency using timeout, circuit breaker and bulkhead. Timeout prevents requests from waiting indefinitely, circuit breaker prevents repeated calls to an unhealthy service, and bulkhead prevents that dependency from consuming all application resources."

Scenario 5: 500-line method with many if/else conditions

Question:

How would you refactor it?

I would **not** rewrite the entire method immediately.

First I would understand what responsibilities it contains.

Suppose:

</> Java



```
processOrder() {  
    if (type == A) {  
        ...  
    } else if (type == B) {  
        ...  
    } else if (type == C) {  
        ...  
    }  
  
    // validation  
    // payment  
    // notification  
    // discount  
    // shipping  
}
```

This method probably violates **Single Responsibility Principle**.

Step 1: Identify responsibilities

Extract:

Validation
Payment
Discount
Shipping
Notification
Order processing



Step 2: Extract methods/classes

Instead of:

</> Java



```
processOrder()
```

we could have:

</> Java



```
validateOrder();  
calculateDiscount();  
processPayment();  
createShipment();  
sendNotification();
```

Step 3: Replace large if/else

If behavior varies by type:

</> Java



```
interface PaymentProcessor {  
    void process(Payment payment);  
}
```

Implementations:

</> Java



```
class CardPaymentProcessor implements PaymentProcessor  
class UpiPaymentProcessor implements PaymentProcessor  
class WalletPaymentProcessor implements PaymentProcessor
```

Then use a strategy/factory approach.

Step 4: Make changes incrementally

For production code, I would:

Add tests



Extract small responsibility



Run tests



Extract next responsibility



Refactor conditionals



I wouldn't perform a huge rewrite without tests.

Strong answer

"I would first identify the responsibilities and business rules hidden inside the 500-line method. Then I'd extract cohesive responsibilities, use Strategy/Factory where behavior varies by type, reduce nested conditionals, and add tests before and during the refactoring. My goal would be to improve maintainability without changing behavior."

Scenario 6: Occasional OutOfMemoryError

Question:

Ignore JVM/memory-management internals. How would you investigate from the application side?

Since we're deliberately ignoring JVM internals, I'd investigate **application behavior and object growth**.

Check 1: Large collections

For example:

</> Java



```
private static final List<Order> orders = new ArrayList<>();
```

If entries continuously accumulate:

```
Order 1  
Order 2  
Order 3  
...  
Millions
```



the application keeps references.

Check 2: Unbounded caches

</> Java



```
Map<String, Object> cache = new HashMap<>();
```

If there is no eviction policy:

cache → unlimited growth



Use an appropriately bounded cache when caching is actually needed.

Check 3: Static references

Look for:

```
</> Java
```



```
static List<Object>  
static Map<Object, Object>
```

especially collections that continuously grow.

Check 4: Large request/response objects

For example:

```
</> Java
```



```
List<Customer> customers = repository.findAll();
```

If millions of records are loaded at once, application behavior itself is problematic.

Prefer pagination/streaming/batching where appropriate.

Check 5: File/resource handling

Look for application-level leaks:

```
</> Java
```



```
InputStream  
Connection
```

`ResultSet`

Use try-with-resources:

`</> Java`

```
try (InputStream input = ...) {  
    ...  
}
```



Check 6: Production evidence

I would correlate:

```
OOM occurrence  
  ↓  
specific endpoint  
  ↓  
request size  
  ↓  
traffic pattern  
  ↓  
recent deployment  
  ↓  
specific code path
```



Strong answer

"From the application side I'd look for unbounded collections/caches, static references, large result sets, large payloads, resource leaks, and objects retained by long-lived application components. I'd correlate the OOM events with endpoints, traffic patterns, deployments and specific application workflows."

Scenario 7: Production API occasionally hangs

Question:

What would you check?

"Occasionally" is an important clue.

I would investigate whether requests are:

```
waiting for DB
waiting for downstream API
waiting for connection
waiting for thread
waiting for lock
```



Check thread behavior

Look for:

- Thread pool exhaustion
- Blocked threads
- Waiting threads
- Deadlocks
- Lock contention

Check DB

- Connection pool exhausted?
- Slow query?
- DB connection timeout?
- Transaction waiting?

Check downstream services

```
API
↓
Service A
↓
Service B ← hanging
```



Does Service B have a timeout?

If not, requests could remain blocked.

Check connection pools

For example:

```
Max DB connections = 50
Active = 50
Waiting requests = 200
```



The application might appear to "hang" while actually waiting for connections.

Check logs/tracing

I would find:

```
request started  
request finished
```



and identify requests where the finish event is missing/delayed.

Strong answer

"For intermittent hangs, I would check thread dumps/blocked threads, DB connection pool exhaustion, slow queries, downstream calls without proper timeouts, lock contention, deadlocks, and HTTP connection pool exhaustion. Distributed tracing is especially useful because it tells me exactly which span the request is waiting on."

Scenario 8: Application has 100 threads but throughput is poor

Question:

What could be wrong?

This is a classic trap.

More threads $\neq$ more throughput.

Possible bottlenecks:

1. Database

```
100 threads  
↓  
DB supports only 20 concurrent connections
```



80 threads may simply wait.

2. External service

100 threads



slow API



Threads spend their time blocked.

3. Lock contention

</> Java

synchronized



If many threads need the same lock:

Thread 1 → executing

Thread 2 → waiting

Thread 3 → waiting

Thread 4 → waiting



4. CPU bottleneck

If the application is CPU-bound, throwing more threads at it can increase context switching without improving throughput.

5. Queue bottleneck

Maybe requests are waiting in:

Thread Pool



Queue



Database



6. Incorrect thread-pool configuration

For example:

```
100 application threads  
20 DB connections
```



The DB becomes the bottleneck.

Strong answer

"I would identify the actual bottleneck before increasing thread count. It could be DB connections, downstream latency, lock contention, CPU saturation, queueing, or an undersized dependency pool. I would use metrics, profiling and thread dumps to determine whether threads are running or mostly waiting."

Scenario 9: Three independent APIs must be called

Question:

How would you make the calls concurrently?

This is a good **CompletableFuture** question.

Sequential implementation:

</> Java



```
A a = callA();  
B b = callB();  
C c = callC();
```

Potentially:

```
A = 500ms  
B = 700ms  
C = 300ms
```

```
Total ≈ 1500ms
```



If independent, execute concurrently.

</> Java



```
CompletableFuture<A> futureA =  
    CompletableFuture.supplyAsync(() -> callA(), executor);  
  
CompletableFuture<B> futureB =  
    CompletableFuture.supplyAsync(() -> callB(), executor);  
  
CompletableFuture<C> futureC =  
    CompletableFuture.supplyAsync(() -> callC(), executor);
```

Then:

```
</> Java   
  
CompletableFuture.allOf(  
    futureA,  
    futureB,  
    futureC  
).join();
```

After completion:

```
</> Java   
  
A a = futureA.join();  
B b = futureB.join();  
C c = futureC.join();
```

Important 5-year point

I would generally prefer a **custom executor** for blocking downstream calls rather than blindly using the common pool.

For example:

```
</> Java   
  
ExecutorService executor =  
    Executors.newFixedThreadPool(20);
```

The exact pool size should be determined from workload, downstream capacity and measurements.

Also handle failures/timeouts

Production code should consider:

API A → success

API B → timeout

API C → success



Should the entire request fail?

Or can we return partial data?

That is a **business decision**, not merely a Java decision.

Scenario 10: One downstream service is unreliable

Question:

How do you prevent cascading failures?

This is one of the most important microservices interview questions.

Suppose:

Service A

↓

Service B

↓

Service C



C becomes slow.

Then:

C slow

↓

B waits

↓

B threads exhausted

↓

A waits

↓

A threads exhausted

↓

Entire system degraded



This is a **cascading failure**.

Solution 1: Timeout

Never wait forever.

Service C timeout = 500 ms



Solution 2: Circuit Breaker

If C keeps failing:

CLOSED
↓
Failures
↓
OPEN



Calls are rejected quickly.

After a recovery period:

OPEN
↓
HALF_OPEN
↓
test request



If successful:

CLOSED



Solution 3: Bulkhead

Isolate resources.

Service A calls
|
+----- C pool → 10 threads
|
+----- D pool → 20 threads



C cannot consume resources belonging to D.

Solution 4: Retry carefully

Retries can actually make the problem worse.

Suppose:

1000 requests



all fail and each retries 3 times:

$1000 \times 3 = 3000$ additional calls



That can overload an already unhealthy service.

So retries should have:

- Small retry count
 - Backoff
 - Jitter
 - Appropriate timeout
 - Idempotency consideration
-

Solution 5: Fallback

If possible:

Service C unavailable



Cached/default/partial response



But don't create fake data where correctness matters.

Solution 6: Rate limiting

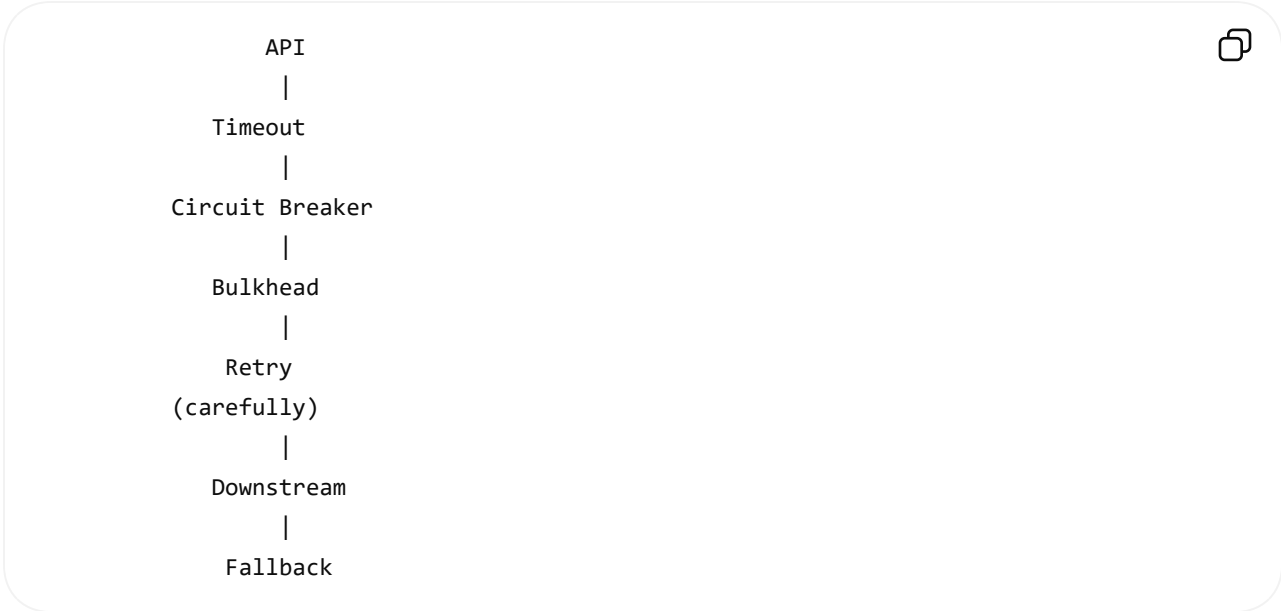
Protect downstream services from excessive traffic.

★ What a 5-Year Candidate Should Say

If the interviewer asks:

"How would you design a resilient Java microservice?"

A strong answer is:



And you should explain the purpose of each:

Mechanism	Purpose
Timeout	Don't wait indefinitely
Circuit Breaker	Stop calling unhealthy service
Bulkhead	Isolate resources
Retry	Recover from transient failures
Backoff/Jitter	Prevent retry storms
Fallback	Graceful degradation
Rate Limiting	Protect capacity
Caching	Reduce repeated dependency calls

The Difference Between 3-Year and 5-Year Answers

A 3-year answer might be:

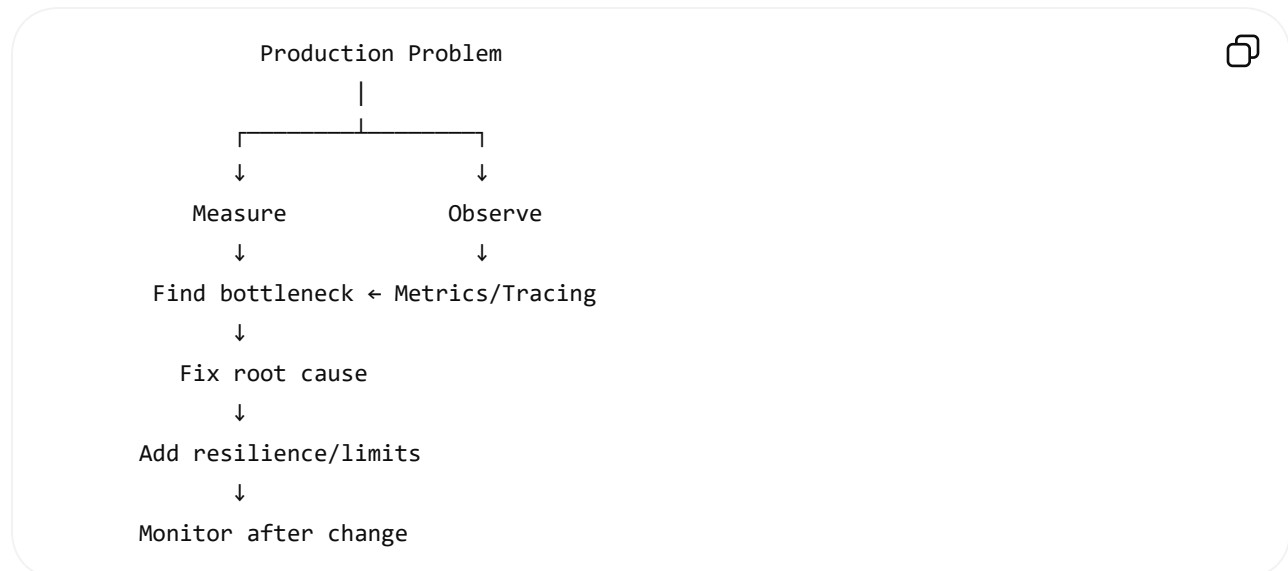
"I'll use `CompletableFuture`."

A 5-year answer should be:

"First I'll identify whether the operations are independent and whether they're CPU-bound or blocking I/O. If they're independent blocking calls, I'll execute them concurrently using an appropriately sized executor, add timeouts, handle partial failures, and ensure the concurrency doesn't exceed downstream capacity."

That's the level interviewers generally expect.

The recurring mindset is:



Don't jump directly to "add more threads", "add caching", or "use async." First explain what evidence tells you that particular solution is appropriate.



Your API suddenly became slow. How do you investigate?
CPU is normal but response time increased. What do you check?

Database is slow. How do you determine whether Java or DB is the bottleneck?

Thread pool is exhausted. What could cause it?

Connection pool is exhausted. What could cause it?
API has high latency but low CPU. Why?
How do you identify slow endpoints?
How do you optimize a slow REST API?

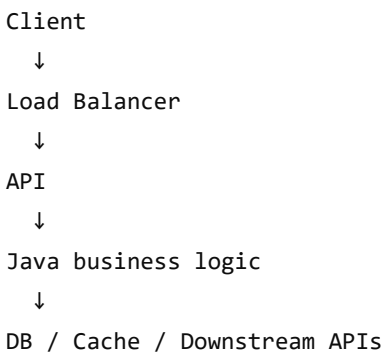
Show more ▾

Yes — these are **very important 5-year Java/Spring Boot production questions**. The key is to answer them like someone who has actually debugged production systems, not just someone who knows definitions.

1. Your API suddenly became slow. How do you investigate?

I would **not immediately increase CPU, threads, or instances**. First I would identify where the latency is coming from.

My approach:



```
graph TD; Client --> LB[Load Balancer]; LB --> API; API --> JBL[Java business logic]; JBL --> DB[DB / Cache / Downstream APIs];
```

Step 1 — Check metrics

I would compare the application **before and after** the slowdown:

- Request rate / throughput
- p50 latency
- p95 latency
- p99 latency
- Error rate
- CPU
- Memory
- Thread-pool utilization
- DB connection-pool utilization

- HTTP connection-pool utilization

Step 2 — Use distributed tracing

For example:

API request: 2 seconds



Controller	20 ms	
Business logic	50 ms	
Database	1.5 sec	← bottleneck
Service B	300 ms	
Serialization	30 ms	

Now I know where to investigate.

Step 3 — Check recent changes

Since it **suddenly** became slow, I would check:

- Recent deployment
- SQL changes
- Configuration changes
- New downstream API
- Feature flags
- Infrastructure changes
- Increased traffic

Step 4 — Fix the actual bottleneck

I wouldn't say:

"I'll increase the thread pool."

I'd say:

"I'll first identify whether we're blocked on DB, downstream calls, connection pools, locks, or application processing and then optimize that specific bottleneck."

2. CPU is normal but response time increased. What do you check?

This is a very common interview scenario.

Normal CPU does not mean the application is healthy.

The threads may simply be **waiting**.

I would check:

Database

Slow SQL
Connection pool exhausted
DB locks
DB overloaded



Downstream APIs

Java Service
↓
External Service
↓
2 second response



Thread pool

Maybe:

100 threads
90 WAITING
10 RUNNING



CPU can remain low.

Connection pools

Check:

- Active connections
- Idle connections
- Pending requests
- Connection acquisition time
- Timeout count

Locks

Threads may be blocked on:

</> Java



synchronized
Lock
ReentrantLock

Queues

Requests may be waiting in an executor or internal queue.

Strong answer

"If CPU is normal but latency increases, I'd focus on waiting rather than CPU consumption—database latency, downstream services, connection pools, thread pools, locks, queues and network I/O."

3. Database is slow. How do you determine whether Java or DB is the bottleneck?

This is a very good 5-year question.

I would use timing and tracing.

Suppose:

Total API latency = 1500 ms



Break it down:

Controller	10 ms
Business logic	40 ms
DB connection acquire	20 ms
SQL execution	1400 ms
Response mapping	30 ms



Clearly DB/SQL is the bottleneck.

But suppose:

Controller	10 ms
Business logic	900 ms



DB execution	50 ms
Response mapping	20 ms

Then Java/application processing is the problem.

What would I check on the Java side?

- Time spent executing the query
- Connection acquisition time
- Number of DB calls
- Transaction duration
- ORM-generated SQL
- N+1 queries
- Result-set size
- Object mapping time

For example:

One API request



100 DB queries

x

10 ms each

= potentially huge latency

What would I check on DB side?

- Query execution time
- Execution plan
- Missing indexes
- Full table scans
- Lock contention
- DB CPU
- DB connections
- Slow queries
- Query frequency

Strong answer

"I would separate connection-acquisition time, SQL execution time, and Java processing time using metrics or tracing. Then I'd compare that with database-side query metrics and execution plans. That tells me whether Java is waiting on the DB or the DB itself is executing inefficiently."

4. Thread pool is exhausted. What could cause it?

Thread-pool exhaustion means the available workers are occupied and new work has to wait or be rejected.

Common causes:

1. Slow downstream API

100 threads



all waiting for external API



2. Slow DB

Threads



waiting for DB connections/results



3. Blocking operations

For example:

</> Java

```
future.get();
```



or blocking HTTP/file/network operations.

4. Deadlock / lock contention

Threads may be waiting for locks.

5. Incorrect pool size

Too few threads for the workload.

But **blindly increasing the pool is dangerous.**

You could turn:

100 waiting threads



into:

500 waiting threads



and overload the DB/downstream service.

6. Long-running tasks

One task may hold a worker for a very long time.

What I check

- Active threads
- Queue size
- Task execution time
- Rejected tasks
- Thread dumps
- Blocking calls
- Downstream latency
- DB pool utilization

5. Connection pool is exhausted. What could cause it?

This could refer to a DB connection pool or HTTP client connection pool.

For DB connections, common causes:

1. Connections aren't returned

For example, incorrect resource handling.

Use:

</> Java



```
try (Connection connection = dataSource.getConnection()) {  
    ...  
}
```

2. Long-running queries

Connection acquired
↓
SQL takes 10 seconds
↓
connection remains occupied



3. Long transactions

A transaction keeps a connection longer than necessary.

4. Too many concurrent requests

For example:

1000 requests/sec
↓
DB pool = 20



Many requests may wait.

5. DB is slow

The connection isn't leaked; it's simply being held for too long.

6. Pool is undersized

But again:

Increasing pool size isn't always the solution.

If the DB can efficiently handle only 50 concurrent operations, increasing the application pool from 50 → 200 may make DB performance worse.

What I would investigate

Connection acquisition time
Active connections
Idle connections
Pending requests



Connection usage duration

Slow SQL

Transaction duration

6. API has high latency but low CPU. Why?

Because latency isn't the same thing as CPU processing time.

The application could be waiting.

Typical reasons:

High latency

```
|
+-- DB
|
+-- External API
|
+-- Network
|
+-- Connection pool
|
+-- Thread pool
|
+-- Lock
|
+-- Queue
|
+-- Disk/I/O
```



Example:

Request = 2 seconds

CPU execution = 50 ms

DB wait = 1.2 sec

External API = 700 ms



CPU remains low while latency is high.

Interview statement

"Low CPU with high latency usually makes me investigate I/O and waiting: database,

downstream services, connection pools, thread pools, locks and network calls."

7. How do you identify slow endpoints?

I would use **endpoint-level latency metrics**.

For example:

```
GET /users      p95 = 100 ms
GET /orders     p95 = 800 ms
GET /payments   p95 = 2.5 sec
```



Immediately `/payments` deserves investigation.

I would monitor:

- Request count
- Average latency
- p50
- p95
- p99
- Error rate

Why p95/p99?

Average can hide outliers.

Suppose:

```
99 requests = 100 ms
1 request   = 10 sec
```



Average may not tell the whole story.

p99 exposes the tail latency.

Then trace the endpoint

```
POST /orders
|
+-- DB = 1.2 sec
```



```
|  
+-- Payment API = 500 ms  
|  
+-- Java = 50 ms
```

8. How do you optimize a slow REST API?

I follow:

```
Measure  
↓  
Find bottleneck  
↓  
Optimize  
↓  
Measure again
```



Possible optimizations:

Database

- Optimize SQL
- Add appropriate indexes
- Avoid N+1 queries
- Fetch only required columns
- Pagination
- Batch operations

Application

- Remove unnecessary computation
- Avoid repeated calls
- Reduce object creation where it actually matters
- Optimize serialization
- Improve algorithms

External calls

- Parallelize independent calls
- Set timeouts
- Connection pooling
- Caching where appropriate

Response

Avoid unnecessarily large payloads.

Instead of:

</> JSON



```
{  
  "user": "...",  
  "orders": "...",  
  "history": "...",  
  "everything": "..."  
}
```

return only what the API contract requires.

Caching

If data is repeatedly requested and can tolerate caching, cache it.

9. How do you reduce unnecessary DB calls?

This is extremely important in Spring Boot/JPA interviews.

Suppose:

API

↓

Get Order

↓

Get Customer

↓

Get Address

↓

Get Product

↓

Get Payment



You might accidentally execute many DB queries.

Techniques

1. Avoid N+1

Instead of:

```
1 query for orders
+
100 queries for customers
```



design the query/fetch strategy appropriately.

2. Fetch only what you need

Don't load a huge entity graph when you only need:

```
id
name
status
```



Use projections/DTO queries where appropriate.

3. Batch operations

Instead of:

```
INSERT
INSERT
INSERT
INSERT
...
```



use batching when appropriate.

4. Cache frequently accessed data

For example:

```
Country codes
Product metadata
Configuration
```



5. Reuse data within a request

Don't repeatedly call:

```
</> Java
```



```
customerRepository.findById(id);
```

five times during the same request if the same result can safely be reused.

6. Analyze tracing

A trace can reveal:

```
GET /orders
```

```
DB call #1
```

```
DB call #2
```

```
DB call #3
```

```
...
```

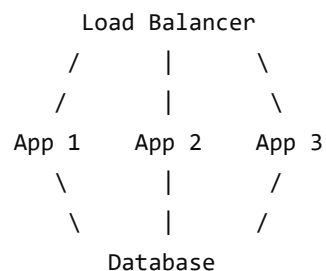
```
DB call #120
```

That's a huge clue.

10. How do you handle high traffic?

I would think about **capacity, bottlenecks and horizontal scaling**.

Architecture:



1. Make services stateless

Then we can add instances:

```
3 instances → 10 instances
```

without relying on local session state.

2. Load balancing

Distribute traffic across instances.

3. Caching

Reduce repeated DB/downstream work.

4. Database optimization

Because scaling Java instances doesn't help if:

Java → DB ← bottleneck



5. Async processing

For work that doesn't need to happen before returning the response:

API
↓
Queue
↓
Worker



6. Rate limiting

Protect the system from excessive traffic.

7. Backpressure

Don't allow incoming work to grow indefinitely when downstream capacity is limited.

Strong answer

"I would first determine the current bottleneck and capacity limits. Then I'd use horizontal scaling for stateless application instances, caching, DB optimization, asynchronous processing where appropriate, rate limiting and backpressure. I would also load-test the system rather than assuming more instances automatically increase throughput."

11. How would you implement caching?

In Spring Boot, a common approach is Spring Cache abstraction.

For example:

 </> **Java**

```
@Cacheable("products")
public Product getProduct(Long id) {
    return repository.findById(id)
        .orElseThrow();
}
```

First request:



```
API
↓
Cache miss
↓
DB
↓
Cache
↓
Response
```

Next request:



```
API
↓
Cache hit
↓
Response
```

This avoids the DB call.

Important caching decisions

You need to define:

TTL

How long is cached data valid?

 TTL = 5 minutes

Eviction

When should cache entries be removed?

For example after an update:

```
UPDATE product
  ↓
evict/update cache
```



Cache-aside

Very common:

```
Read
  ↓
Cache?
├─ hit → return
└─ miss
    ↓
    DB
    ↓
    populate cache
```



What should NOT blindly be cached?

Things such as highly dynamic or highly sensitive data require careful consideration.

12. Where would you cache?

There are several levels.

```
Client
  ↓
CDN / Edge
  ↓
Application cache
  ↓
Distributed cache
  ↓
Database
```



But each level solves a different problem.

Client-side cache

Useful for suitable client data.

CDN

Excellent for:

- Static files
- Images
- JS/CSS
- Cacheable HTTP responses

Application/local cache

Inside the Java process.

App instance
↓
Local memory



Very fast, but each instance has its own cache.

Distributed cache

For example:

App 1 ↘
App 2 ┴→ Redis
App 3 ↙



All application instances can share cached data.

13. Local cache vs distributed cache

Local cache

Example:

App 1 → local cache
App 2 → local cache
App 3 → local cache



Advantages

- Extremely fast
- No network hop

- Simple
- Low latency

Disadvantages

Each instance has its own copy.

App 1 → user=John
App 2 → user=OldValue



Consistency becomes a concern.

If you have 20 instances, the same data might exist 20 times.

Distributed cache

App 1 ↗
App 2 ┼→ Redis
App 3 ↘



Advantages

- Shared cache
- Better consistency across instances
- Centralized cache
- Useful in horizontally scaled applications

Disadvantages

- Network call
- Redis/cache infrastructure required
- Network failures
- Serialization/deserialization overhead
- Cache availability becomes another concern

14. Redis vs in-memory cache

This is basically a practical comparison.

	In-memory	Redis
Location	JVM	Separate server/cluster
Speed	Extremely fast	Very fast
Shared between instances	✗	✓
Network call	✗	✓
Data survives JVM restart	Usually ✗	Depends on Redis configuration
Scalability	Per application instance	Centralized/distributed
Consistency	More difficult	Easier to share
Infrastructure	Simple	Additional infrastructure

Example

If you have:

3 Spring Boot instances



and use local cache:

App1 → Cache A
App2 → Cache B
App3 → Cache C



With Redis:

App1 ↙
App2 ──→ Redis
App3 ↘



Interview answer

"I'd choose a local cache when extremely low latency is important and stale or duplicated data across instances is acceptable. I'd choose Redis when multiple

application instances need a shared cache or when centralized cache management is more appropriate."

15. What happens if cache goes down?

This is where a 5-year answer differs from a basic answer.

The application should ideally **degrade gracefully** rather than **completely fail**.

For cache-aside:

Request



Cache



CACHE DOWN



DB



Response



The cache should be treated as an **optimization**, not necessarily the system of record.

But there's a danger

Suppose:

1000 requests/sec



Normally:

900 → Cache

100 → DB



Cache goes down:

1000 → DB



Now the DB could become overloaded.

This is called a **cache stampede / fallback load problem**, depending on the exact pattern.

So I would consider:

- Timeouts for cache access
- Fail-fast behavior
- Circuit breaker where appropriate
- DB capacity
- Rate limiting
- Request coalescing/single-flight patterns
- Graceful degradation
- Cache redundancy/HA

Very important distinction

If the cache contains merely derived/retrievable data:

Cache down → fetch from DB



is reasonable.

But if the application incorrectly treats the cache as the **only source of truth**, cache failure can become a major outage.

One Production Scenario Connecting All 15 Questions

Imagine an interviewer says:

"We deployed a new version. Suddenly /orders p99 went from 300 ms to 5 seconds. CPU is only 30%. What do you do?"

A strong 5-year candidate might walk through:

1. Check p50/p95/p99
- ↓
2. Compare before/after deployment
- ↓
3. Check distributed traces
- ↓
4. Identify DB/downstream latency
- ↓
5. Check DB connection pool
- ↓
6. Check thread pool



↓

7. Check SQL count
- ↓
8. Look for N+1 queries
- ↓
9. Check recent code/config changes
- ↓
10. Fix root cause
- ↓
11. Load test
- ↓
12. Monitor production

Suppose tracing reveals:

/orders = 5 sec



Java processing	50 ms	
Redis	10 ms	
DB connection	30 ms	
DB query	3.8 sec	←
Payment API	900 ms	

Now your investigation becomes:

DB

↓

Execution plan

↓

Missing index?

Slow query?

Lock?

DB overload?



and separately:

Payment API

↓

900 ms

↓

Timeout?

Circuit breaker?

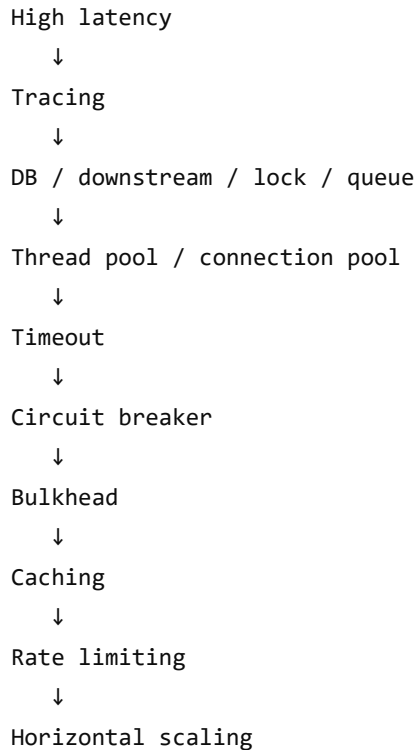
Parallelization?



That is the **5-year mindset**:

Don't treat "API is slow" as one problem. Break the request into measurable components and find where the time is actually being spent.

★ Concepts you should be able to connect in an interview



```
graph TD; A[High latency] --> B[Tracing]; B --> C[DB / downstream / lock / queue]; C --> D[Thread pool / connection pool]; D --> E[Timeout]; E --> F[Circuit breaker]; F --> G[Bulkhead]; G --> H[Caching]; H --> I[Rate limiting]; I --> J[Horizontal scaling];
```

If you can explain these scenarios with **real examples + trade-offs + what metrics you would check**, you're answering at a much stronger 5-year Java/Spring Boot level.



Scenario 1 — Slow API

Your API response time increased from 200 ms to 3 seconds after deployment. CPU is normal. How will you investigate?

Scenario 2 — High Traffic

Your API normally receives 100 requests/sec but suddenly receives 5,000 requests/sec. What happens and how would you handle it?

Scenario 3 — Downstream Failure

[Show more](#) ✓

Scenario 10 — Deployment

Your new version is deployed, but some users are receiving errors while others are fine. What could be happening?

This is a **very good 5-year production troubleshooting question** because the key clue is:

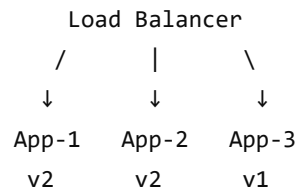
Some users work, while others fail.

That usually means the problem is **not affecting every request equally**. I would investigate from the load-balancer level down to the application instance.

1. Multiple application instances may have different versions

This is one of the first things I would check.

Suppose we have:



If App-1 and App-2 have the new version but App-3 still has the old version, requests routed to different instances can behave differently.

For example:

User A → App-1 → works
User B → App-2 → works
User C → App-3 → old behavior/error

What I check

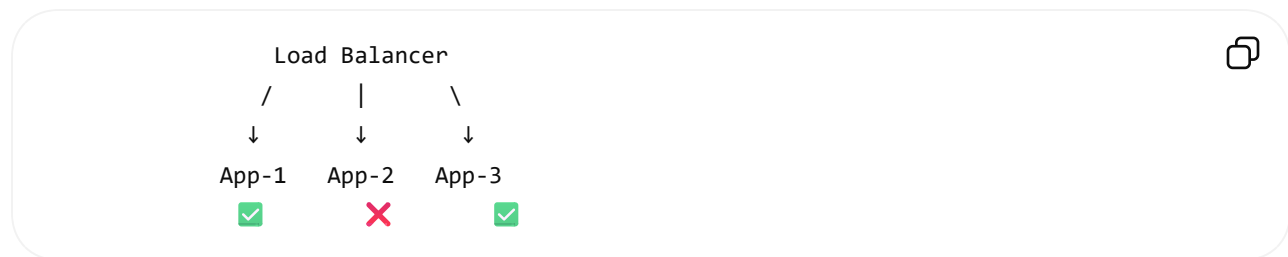
- Deployment status
- Running application version on every instance

- Instance registration with the load balancer
- Health-check status
- Deployment logs

I would ideally expose a safe version/build identifier through monitoring or an internal diagnostic endpoint.

2. One application instance may be unhealthy

Even if all instances have the same version:

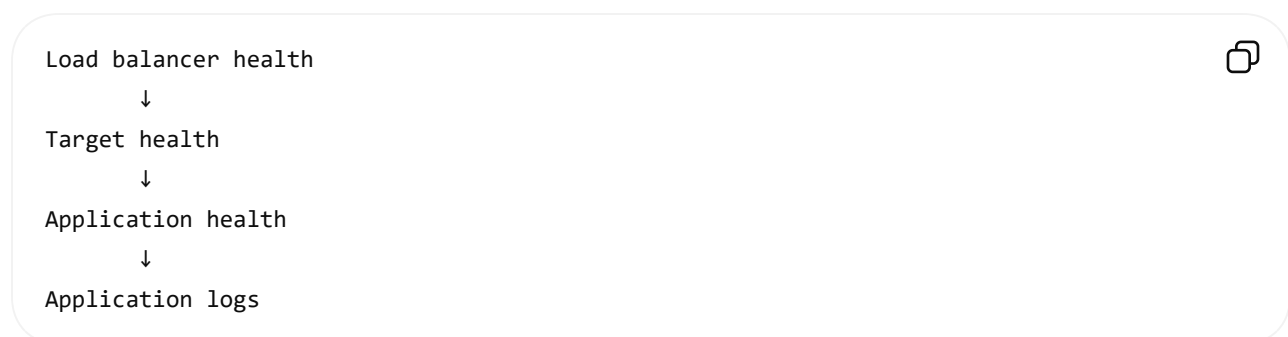


App-2 might have:

- Failed initialization
- Broken configuration
- Connection problem
- Dependency failure
- Different environment variable
- Corrupted local state

The load balancer may still send traffic to it if its health check isn't detecting the actual problem.

So I'd check:



3. Sticky sessions / session affinity

This is another important possibility.

Suppose the load balancer uses sticky sessions:

```
User A → App-1  
User B → App-2  
User C → App-3
```



If App-2 has a problem, **only users mapped to App-2** may see errors.

This can make the problem appear random:

```
Same API  
Same code  
Same request  
  
User A → works  
User B → error  
User C → works
```



I would check whether session affinity is configured and whether the affected users are consistently routed to one instance.

4. Different configuration between instances

This is very common in production.

For example:

```
App-1:  
PAYMENT_URL=https://payment-v2  
  
App-2:  
PAYMENT_URL=https://payment-v2  
  
App-3:  
PAYMENT_URL=https://payment-old
```



Or:

App-1 → correct DB

App-2 → correct DB

App-3 → wrong DB



Possible differences:

- Environment variables
- Config files
- Secrets
- Database URLs
- Downstream URLs
- Feature flags
- Timeout configuration
- Credentials
- Region/AZ-specific configuration

So I would compare the **effective configuration** of healthy and unhealthy instances.

5. Database schema/version mismatch

This is a particularly important deployment scenario.

Suppose the new application expects:

`orders.payment_status`



but the database migration hasn't been applied everywhere or correctly.

Or imagine:

Application v2



expects new DB schema

Database



old schema



Then requests can fail.

This becomes especially dangerous if application and database changes aren't backward compatible.

Better deployment approach

Use backward-compatible database migrations:

```
graph TD; A[Old App] --> B[New DB schema]; B --> C[New App];
```

Old App
↓
New DB schema
↓
New App



rather than:

```
graph TD; A[Old App] --> B[Immediately incompatible DB schema]; B --> C[New App];
```

Old App
↓
Immediately incompatible DB schema
↓
New App



6. Canary deployment

Maybe the new version was intentionally deployed only to a percentage of instances.

For example:

```
graph TD; LB[Load Balancer] -->|90%| v1[v1]; LB -->|10%| v2[v2];
```

Load Balancer
/ \
↓ ↓
v1 90% v2 10%



If v2 has a bug:

Most users → v1 → works
Some users → v2 → error



That's actually one reason canary deployments are useful: you can detect problems before moving everyone to the new version.

7. Feature flag problem

The new deployment may have introduced a feature flag:

```
if (newPaymentFlowEnabled) {  
    newImplementation();  
} else {  
    oldImplementation();  
}
```



If the flag is enabled only for certain:

- Users
- Regions
- Tenants
- Accounts
- Customers

then only some users may experience the problem.

For example:

```
User A → flag OFF → old flow → works  
User B → flag ON  → new flow → error
```



So I'd check feature-flag configuration and the affected users' characteristics.

8. Cache inconsistency

Suppose there is a local cache:

```
App-1 → Cache contains new configuration  
App-2 → Cache contains old configuration  
App-3 → Cache contains old configuration
```



Then users routed to different instances can see different behavior.

This is particularly relevant when configuration or data is cached locally.

I'd check:

- Cache contents

- Cache TTL
- Cache invalidation
- Whether all instances were restarted
- Whether distributed cache is being used

9. Load balancer routing issue

I'd also verify whether the load balancer is distributing traffic correctly.

For example:

App-1 → 10%

App-2 → 10%

App-3 → 80%



If App-3 is unhealthy or overloaded, a large percentage of users can fail.

I'd check:

- Target distribution
- Health checks
- 4xx/5xx per instance
- Response time per instance
- Connection errors

10. Version compatibility between services

Suppose:

Order Service v2

↓

Payment Service v1



The new Order Service might send:

</> **JSON**

```
{  
  "paymentMethod": "UPI",
```



```
"newField": "..."  
}
```

while the old Payment Service expects a different contract.

Or the reverse:

Order v1 → Payment v2

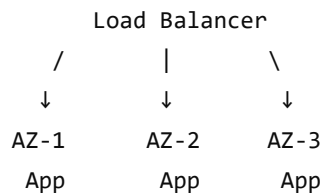


This can cause failures for only requests routed through a particular combination of service versions.

This is why **backward-compatible API contracts** are important during rolling deployments.

11. Network/AZ-specific problem

If your application is deployed across availability zones:



A networking/configuration issue in one AZ could cause:

```
AZ-1 → healthy  
AZ-2 → healthy  
AZ-3 → errors
```



I'd compare error rates and latency by:

- Instance
- Availability zone
- Region

How I would investigate in production

I would follow this sequence:



```
graph TD; A[Users reporting intermittent errors] --> B[Check error rate]; B --> C[Isolate affected requests]; C --> D[Compare healthy vs failing requests]; D --> E[Check load balancer]; E --> F[Check which instance handled request]; F --> G[Compare application versions]; G --> H[Compare configuration]; H --> I[Check logs + distributed traces]; I --> J[Check DB/schema version]; J --> K[Check downstream service versions]; K --> L[Check feature flags]; L --> M[Check cache]; M --> N[Check AZ/region];
```

Users reporting intermittent errors

↓

Check error rate

↓

Isolate affected requests

↓

Compare healthy vs failing requests

↓

Check load balancer

↓

Check which instance handled request

↓

Compare application versions

↓

Compare configuration

↓

Check logs + distributed traces

↓

Check DB/schema version

↓

Check downstream service versions

↓

Check feature flags

↓

Check cache

↓

Check AZ/region

The **most important thing** is to correlate the failing request with the exact instance that processed it.

For example:



```
Request ID: abc123
User:      X
Status:    500
Instance:  order-service-7f8c
Version:   2.4.1
AZ:        ap-south-1b
```

Then compare it with a successful request:



```
Request ID: xyz789
User:      Y
Status:    200
Instance:  order-service-3a21
```

Version: 2.4.0
AZ: ap-south-1a

Now you have a strong lead:

Failures are occurring only on version 2.4.1.

★ Interview-ready answer

If the interviewer wants a concise answer, I would say:

"Since only some users are affected, I'd first determine whether the requests are being routed to different application instances, versions, configurations, or feature-flag paths. I'd check load-balancer target health, instance-level error rates, deployment status, application version, configuration, logs and distributed traces. I'd also check database schema compatibility, downstream service versions, cache consistency, sticky sessions, and whether the problem is isolated to a particular AZ or region.

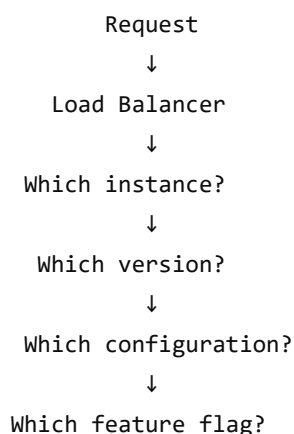
I'd correlate a failing request ID with the exact instance, version and environment that processed it. If the problem is isolated to the new version, I'd stop or roll back the rollout and investigate the deployment before continuing."

🔥 The key 5-year thinking

Don't answer only:

"Maybe load balancer issue."

Think in terms of **request → routing → instance → version → configuration → dependency → data**.



↓
Which DB/dependency?
↓
Exact failure

That troubleshooting chain is what makes the answer sound like a **production-level 5-year Java/Spring Boot engineer** rather than a purely theoretical answer.



ChatGPT can make mistakes. Check important info.